FINEX | Complete solo project blueprint

One developer.
The full handbook scope.
A clear implementation path.

Shivraj Dhote | Full descriptive edition | 16 October 2026

Prepared directly from the newly supplied FinEx Combined Implementation Handbook v1.0 (10 August 2026). All eight sprints, four technical tracks and 96 original sprint task IDs are retained. Every development task is reassigned to Shivraj.

Included	Coverage
Clients	React web and Flutter mobile
Platform/data	FastAPI, operational Lakebase/Postgres, Databricks Delta, contracts, CI/CD
AI and product	Risk, market/portfolio, RAG, paper trading, alerts, goals and spending
Release	Quality, security, permissions, observability, UAT, deployment and handover

Full scope is not reduced to a web-only MVP. This edition restores Flutter, alerts, spending and every handbook track to the implementation backlog. It separately identifies a bounded 10 November demonstration checkpoint so unfinished full-scope work is never described as complete.

Reality check: the source assumes twelve developers and ten days per sprint. One person has only 26 calendar days until 10 November. Full completion by that date cannot be responsibly promised without a verified, substantially complete existing codebase.

01 / How to use this complete blueprint

Your goal and responsibilities

You now own Product/UI, API, Data and AI/ML work. Read the sprint overview, then execute its dependency order. Each track page retains the three source task IDs and adds a practical solo explanation. Treat each card as a bounded issue with evidence, rather than starting all twelve cards simultaneously.

Document navigation

Pages	Contents
3-4	Product scope, original architecture and solo workflow
5-6	Real-data sources and ingestion/lineage
7-10	Database, API, user experience and calculation contracts
11-12	Week 0 and the 10 November checkpoint
13-52	Eight sprints, five pages each: overview plus PRD/API/DAT/AI cards
53-56	Security, deployment, acceptance/handover and references

What is inherited versus newly proposed

Original sprint names, task IDs, feature coverage and evidence expectations come from the supplied handbook. The solo execution order, explanatory guidance, suggested schema/API details and checkpoint plan are planning adaptations. Actual repository completion, cloud access, model keys, quotas and implementation paths remain unverified.

Source conflicts handled explicitly

The handbook says operational records use Lakebase/Postgres even where older card wording says Delta. This blueprint applies that correction within the relevant cards. Delta remains the history/analytics/events/evaluation layer. The document does not silently replace Vector Search, Flutter or cloud workflows with simpler tools and call the original requirement complete.

Review and acceptance when you are alone

Use deliberate self-review, executable tests and consumer checks between your own layers. These do not become independent peer approval. The source names Syona as BA/UAT lead; use the currently designated mentor for formal acceptance, and record any missing review or approved exception honestly. You can implement independently without pretending that absent teammates reviewed the work.

02 / Product definition and complete feature map

FinEx is an educational personal-finance and investing prototype. It combines learning from approved documents, analysis of user-entered holdings and real market observations, simulated practice, goal planning and monitoring. It is not a broker, an investment-adviser service or a guaranteed real-time trading product.

Module	User outcome
Identity and risk	Sign in, record consent, complete profile/questionnaire, understand an approved deterministic risk result.
Market and portfolio	Search instruments, track watchlists, enter holdings, inspect sourced values, allocation and limitations.
Research assistant	Ask supported questions, view citations/history, provide feedback and recognize insufficient evidence.
Paper trading	Create a virtual account, buy/sell under explicit fill rules, see history/positions and reconciled performance.
Alerts and intelligence	Manage rule conditions, read notifications, ask portfolio/scenario questions grounded in authorized metrics.
Goals and spending	Create goals/projections; preview/commit a spending CSV; correct categories and inspect trends.
Operations	Run repeatable workflows, evaluate AI, control permissions, recover data and release supported web/mobile builds.

Full product boundary

Real brokerage execution, direct bank aggregation, public app-store publication and custom domains are outside the source zero-cost prototype scope. Android release artifacts and iOS-compatible source/configuration are distinct from actually publishing to a store. Full FinEx completion means the included handbook tasks pass their gates, not that these excluded commercial features exist.

Core user journeys

Entry → consent/profile → risk result. Search → watchlist → manual portfolio → explanation. Question → retrieved evidence → answer/citations → feedback. Virtual account → order → ledger/positions → learning summary. Alert rule → evaluation → notification/read. Goal → projection/progress. CSV → preview/errors → commit → category correction → monthly trend.

03 / Architecture and the solo working method

React / Flutter → versioned FastAPI → domain services → ports/adapters → operational database, analytics, market, retrieval/model and notification providers. The clients never call a database or hold a cloud/provider secret. Each boundary has a schema, example and test.

Source architecture	Implementation responsibility
React / Flutter	React hosted through approved Azure Static Web Apps pipeline; Flutter Android artifact through approved release process; shared generated contracts.
FastAPI / delivery	Containerized API, registry artifact and Azure Container Apps deployment as specified; record actual account capabilities and cost controls.
Lakebase/Postgres	SQLAlchemy/Alembic, mutable user-owned records, constraints and transactional ledger.
Databricks Delta	History, snapshots, events, features, reconciliation and evaluation data; versioned jobs and quality checks.
AI / retrieval	Replaceable LLMProvider and Retriever interfaces; Databricks serving/Vector Search/MLflow where the chosen account supports them.
Auth / notifications	Google OAuth plus backend JWT lifecycle; in-app records and FCM adapter with failure preservation.

How you implement without four separate developers

Choose one vertical behaviour. Freeze its fields/rules; create the minimum data interface; implement/test the service; expose the API; integrate React; integrate Flutter; then run parity and evidence checks. Keep only one main implementation card in progress, with small documentation/testing tasks alongside it. This limits context switching and unfinished layers.

Existing repository rules

Inspect actual paths before creating files. Keep the approved repository branch/release procedure; do not change shared settings because this plan has a new owner. A small commit and reviewed test evidence are easier to recover than a month-long unverified branch. Use native local tools where practical; build the API container for the specified deployment.

04 / Real data: what it means for each module

Domain	Genuine input and boundary
Market	Actual provider records or permitted official report downloads with source, observation date/time and currency. Daily/end-of-day data is real, but not a live tick feed.
Research/RAG	Real approved filings, issuer factsheets and educational documents with URL, reporting period, page/section and checksum.
Portfolio/risk	Your own entered holdings and responses. They are real user input but not broker-verified account balances; score uses approved rules.
Paper trading	Real recorded quote/snapshot inputs, virtual cash and simulated execution. Fill assumptions must be explicit.
Spending	Only your own authorized CSV/manual entries; mask descriptions and identifiers. Synthetic edge-case files remain test fixtures.
Alerts/goals	Rules and assumptions entered by the user; evaluated against real permitted data or deterministic projections. Scenarios are hypothetical, not forecasts.

Practical acquisition choice

For an Indian-equity demonstration start with five verified instruments and a modest daily-history range. NSE official reports provide downloadable data [R1]; check current format and usage terms [R2]. The source handbook suggests yfinance as a prototype adapter; its documentation flags personal-use constraints [R3]. A public hosted display requires suitable permissions, not merely a working download.

Documents and missing information

SEBI reading material [R5] can support educational answers; issuer reports support company claims. A daily price file cannot supply fees, financial statements or all fund facts. Register a separate approved source or return unavailable. Do not ask the model to fill missing business data.

Real-data fallback rule

A failure shows the last permitted cached real record with its actual timestamp, or unavailable. Never generate random prices. Mocks remain useful for isolated tests and contract-first UI development; they must not be represented as real product data.

05 / Ingestion, freshness and lineage workflow

Step 1 - acquire and register

Use the authorized provider client or permitted manual download. Store source name/URL, retrieved_at UTC, original filename and checksum, run ID and schema version. Avoid undocumented scraping and do not retry around access restrictions. Keep raw files only for the allowed retention period.

Step 2 - normalize and validate

Map provider symbol/exchange to canonical instrument ID. Parse dates and numeric precision explicitly; preserve currency and adjusted/unadjusted basis. Reject malformed records, impossible OHLC relationships or negative volume where applicable. Quarantine rejected rows with safe reasons instead of silently discarding them.

Step 3 - persist idempotently

Use a uniqueness key such as source + instrument + observation/session + adjustment basis. Re-importing the same observation must not duplicate history. Operational quote/cache access and Delta history copies must have clear ownership and reconciliation; delayed analytics should not block an atomic paper order.

Step 4 - expose honest status

Keep observation time separate from ingestion/retrieval time. Represent quality/freshness and delivery origin separately: for example, END_OF_DAY observation with served_from_cache=true. A cache fetch today does not make yesterday price LIVE. Define accepted staleness per use case; trading simulation may reject a quote that a chart can still display.

Step 5 - schedule, monitor and recover

Log started/completed/failed, accepted/rejected counts and last successful run. Bound retries and prevent overlapping duplicates. Weekend/holiday absence differs from a missed expected session. Reprocess a failed source run safely and compare counts/hash/key coverage before promoting the new data.

Verification evidence

Compare at least five UI values/dates with their source records; rerun the same import; simulate outage; demonstrate real cached/unavailable behaviour. Preserve checksums and source versions in the report. Do not call a parser complete because it ran on one hand-edited sample.

06 / Data model and storage boundaries

Use UUID identifiers and UTC event timestamps where appropriate, explicit foreign keys and user ownership on private operational entities. Use Decimal/numeric fields for money and quantity with approved precision. Suggested field names are design guidance; reconcile them with the existing schema.

Entity group	Important fields / store
Identity	users(provider, subject), profile/preferences, consent(version, accepted_at), auth audit. Operational Postgres; approved audit copies in analytics.
Risk	questionnaire/version, questions, answers, assessments(score components, band, rule_version, owner). Operational; evaluation history analytical.
Market/portfolio	instruments/provider mapping; quotes/history(source, observed_at, currency, adjustment); portfolios/holdings(owner, quantity, cost). Holdings/watchlists operational; history/snapshots Delta.
AI/RAG	document registry/chunks(page, source hash, period); operational conversations/messages/feedback(owner); retrieval/evaluation events and corpus/index metadata as designed.
Paper	accounts, orders(request key/fingerprint), trades, ledger, positions. Postgres transaction; performance/reconciliation snapshots Delta.
Alerts	rules, events, notifications, attempts, cooldown/dedupe/status. Operational; run/trigger analytics Delta.
Goals/spending	goals/contributions/assumptions; uploads/transactions/categories/corrections. Operational; progress/monthly summary/insight history Delta.

Constraints that matter

Unique provider-subject identity; unique watchlist item; quantity/date/currency validity; scoped order idempotency; category/transaction provenance; non-orphan citations; goal/formula version. Enforce operational integrity at write time and analytical consistency in DQ jobs. A dashboard query cannot replace a transaction constraint.

Recovery and retention

Document migration order, safe reset for test data, forward-fix strategy and backup/restore. Ledger corrections use traceable entries. Deletion must account for operational, analytical, retrieval and backup policy; do not promise instantaneous removal from every retained copy if the system does not do that.

07 / API contracts and endpoint families

Use the source /api/v1 conventions and a documented envelope with safe error codes and correlation ID. Generated TypeScript/Dart clients must compile from the same frozen OpenAPI. The following families describe intended behaviour, not a claim that endpoints already exist.

Family	Operations and required behaviour
Health / identity	Liveness/readiness; Google callback; users/me; refresh/logout. Verify provider identity and token lifecycle; no client-supplied owner trust.
Onboarding / risk	GET/PUT profile; consent/completion; questions; assessments. Versioned input and deterministic scoring.
Market / portfolio	Search/quotes/history; watchlist items; portfolio/holdings CRUD; summary. Source/date/status, pagination and ownership.
Chat / feedback	Conversation/message list/create; supported delete; question orchestration; feedback. Owner scope, citations, bounded timeout and evidence fallback.
Paper	Account create/reset, order submit/list/get, positions/ledger. Request fingerprint, stable replay and atomic effects.
Alerts / intelligence	Rule CRUD/evaluate; notifications read/mark-all; metrics/scenarios. Dedupe/cooldown and authorized tool context.
Goals / spending	Goal CRUD/project/progress; CSV preview/commit; category correction; monthly summaries. Bounded files, validation and privacy.

Every contract includes

Types, requiredness, enums, units/currency, timestamp meaning, ownership, error codes, pagination and normal/empty/boundary/invalid/unauthorized/provider-failure examples. A change that affects clients increments the agreed version and updates examples before integration.

Business rules live server-side

Risk scores, valuation, fills, cash, goals, alerts and privacy decisions are service responsibilities. UI code renders results. The model may explain an approved tool result but must not invent new numbers. No client/provider credential is included in generated API code.

08 / React and Flutter user experience

Shared information architecture

Entry/onboarding, dashboard, market/watchlist, portfolio, chat, paper account, alerts, goals and spending. Use a clear main navigation with private-route guards. Implement the same user goal and contract meaning across platforms; layouts can be adapted to mobile.

Screen	Required content and user states
Dashboard	Portfolio summary, source/as-of label, risk summary and navigation. Empty account is an invitation to enter holdings, not a zero-valued completed portfolio.
Market / holdings	Search, quote/source/time/status, watchlist and validated holding forms; partial valuation warning where needed.
Chat	History/composer, pending/answer/refusal/fallback, inspectable citations, feedback and deletion confirmation.
Paper	Virtual-money label, price/fill policy, ticket, pending/unknown/rejected/filled, ledger/history/positions.
Alerts	Rule editor, cooldown/last evaluation, unread/read notifications and best-effort delivery state.
Goals / spending	Assumption-labelled projections; file requirements and preview errors; explicit commit; masked descriptions and category correction.

Implementation order for each feature

Create fixture/example states → implement reusable UI components → map generated client → connect real API → check persistence and error recovery → run parity on the same test account. Label mock-only states during development. Native mobile cannot be claimed complete using a screenshot of responsive web.

Accessibility and mobile details

Use readable labels, keyboard/focus order, text equivalents for charts and explicit validation messages. On Flutter check keyboard occlusion, safe storage, back/resume and supported file picker. Never remove source/status/disclaimer fields to make the small-screen layout fit.

09 / Deterministic calculations and examples

Risk

Load approved questionnaire weights, willingness/capacity rules and band boundaries from versioned configuration. Keep input answers and output components traceable. Do not choose thresholds simply to finish the feature. Same answers and rule version must reproduce the same result; AI explanation cannot change it.

Portfolio

For one valid same-currency holding: market_value = quantity × accepted price. Total value is the sum of fully supported holdings; weights are value/total when the denominator is valid. Explain partial coverage. Unrealized difference against recorded cost is not automatically total return: fees, dividends, trades and corporate actions affect broader performance.

Goal projection

For an explicitly chosen end-of-month contribution model, monthly effective rate i = (1 + annual_rate)^(1/12) - 1. Future value after n months is P(1+i)^n + C((1+i)^n - 1)/i. At i=0 use P+nC. Other timing conventions require a different formula; store the selected convention, units and version. This is a proposed calculation contract, not an approved expected-return assumption.

Paper ledger example - synthetic test only

Initial virtual cash 10,000; fixture fill 100/unit; no fees. Buy 2: cash 9,800, quantity 2. Replay the same request: unchanged. Sell 1: cash 9,900, quantity 1. An over-sell fails without changes. A conflict key with a different payload must be rejected. These values test integrity, not market performance.

Scenarios, spending and categorization

A hypothetical sector shock uses verified exposure and an explicit scenario percentage; it is not a prediction. Monthly spending sums committed classified rows under clear income/refund/transfer/currency policies. Category confidence needs interpretation and correction; a model score is not automatically a calibrated probability.

Date and adjustment discipline

Do not mix adjusted prices with unadjusted holdings unnoticed. Return history and volatility require enough valid observations and stated methodology. Backtests/scenarios must not use future observations or documents unavailable at their decision time.

10 / Week 0: solo enablement checklist

All source bootstrap responsibilities transfer to Shivraj. Preserve their IDs for traceability; the suffix names identify historical cards, not current owners. Check existing code first and reuse only verified working components.

Source ID	Your deliverable
W0-KHY / W0-NIL	React route/design shell and reusable Button/Input/Card/Modal/Loading/Error UI kit.
W0-SHI / W0-LIK	Flutter flavors/navigation/theme and reusable widgets/state screens; actual supported-target run.
W0-DIS	FastAPI factory, health/readiness, safe errors and passing tests.
W0-NAY	Ports for LLM, retrieval, market, operational/analytics storage.
W0-PAL	Postgres connection, migration baseline and transaction proof.
W0-MIT	Databricks bundle/schema/job skeleton and a quality-checked test Delta write.
W0-PIY	OpenAPI validation, TS/Dart generation and mock-server workflow.
W0-PAY / W0-PRA / W0-MRU	Structured AI validation, policy/prompt registry, controlled mock adapter and initial evaluation runner.
W0-SYO	Glossary, task/acceptance templates, dependency tracking and severity definitions. Formal UAT still belongs to the designated acceptance authority.

Exit proof

Run both clients, API tests, a migration, generated clients and a thin API-to-operational-data sample. Validate the bundle and show a test Delta job result. Record access blockers and actual account capabilities. A local fixture test does not prove cloud service access.

How to timebox it

Use 16-18 October to inventory/recover existing foundations, not to guarantee a complete new Week 0 plus Sprint 1 from scratch. If the baseline is empty/broken, retain the full backlog and reduce the November demonstration claim. Do not pretend setup was completed during the previous week.

11 / Deadline checkpoint versus full completion

The complete plan contains 96 sprint cards plus Week 0. The source ten-day cadence assumed twelve parallel developers. Reassigning their names to one person does not preserve that delivery speed. This full plan therefore uses acceptance-driven sprint order; dates below target a limited demonstration, not all-card completion.

Window	Checkpoint target if prerequisites are already usable
16-18 Oct	Inventory/repair Week 0 and S1; show API, data and client baseline.
19-21 Oct	S2 identity/consent and approved deterministic risk path.
22-25 Oct	S3 real-data ingestion, watchlist and portfolio; verify sources and values.
26-30 Oct	S4 bounded real-document retrieval/cited assistant and safety evidence.
31 Oct-2 Nov	S5 safe paper slice only if reusable foundations exist; otherwise fix core defects. Freeze demo scope.
3-10 Nov	S8-style hardening of included scope, deployment/rehearsal/evidence and submission.
After checkpoint	Finish incomplete S1-S5 cards, full S6/S7 features and complete S8 regression across all features. Full scope remains in the plan.

Full-project forecasting

After the baseline audit, estimate each remaining card in focused hours and add integration/rework reserve. For illustration only, even an average 4-8 hours for each of 96 unfinished cards gives 384-768 hours before Week 0 and additional integration. Your actual code reuse may lower this; unfamiliar Flutter/cloud/security work may increase it. Reforecast after each accepted sprint rather than promise an unsupported final date.

What you can claim on 10 November

List exactly which tasks are integrated, which use cached real observations, which are incomplete and which are deferred. Demonstrate the accepted slice with true source dates. Do not claim the full source handbook finished if alerts, spending, Flutter parity or later acceptance checks remain outstanding.

Sprint 1 / Platform foundation

Implementation owner: Shivraj Dhote. **Source:** Section 6.1 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

Create common contracts, runnable web/mobile shells, operational and analytics foundations, and an initial AI evaluation baseline.

Recommended solo dependency order

Confirm W0 → API conventions and data naming → migrations/adapters → React/Flutter shells and state contract → initial RAG/evaluation → integrated gate.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Both clients and API start; migrations and a Delta query/job work; generated clients compile; CI and evaluation baseline pass.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 1 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S1-PRD-01

Build: Create the React application shell: routes /login, /onboarding, /dashboard, /portfolio, /chat, /paper-trading, /alerts, /goals and /spending; add protected- route placeholder and shared layout.

How to implement alone: Create the route map first, then reuse one layout and shared components across pages. Keep placeholder routes visibly unavailable until their feature is integrated. Test direct URL refresh and narrow/wide layouts; do not duplicate backend calculations in components.

Deliver: PR containing src/app/router.tsx, AppShell, route tests, and screenshots at 360px and 1440px. **Pass when:** All routes render without console errors; direct refresh works; protected routes redirect to /login using mock auth.

S1-PRD-02

Build: Create the Flutter application shell with dev/staging/prod flavors, navigation routes matching the React route map, shared theme, AppButton, AppTextField, AppCard and LoadingView.

How to implement alone: Pin the SDK and first run Android on a supported emulator/device. Build matching navigation and reusable widgets before feature screens. Separate public API base URLs by environment; never bundle provider keys. Record the actual run/build command.

Deliver: Flutter PR with lib/core/theme, lib/core/router, lib/shared/widgets, widget tests and Android debug build evidence. **Pass when:** flutter analyze has zero errors; app opens on Android emulator; every shared widget has loading/disabled/error example.

S1-PRD-03

Build: Create the cross-platform UI contract: field names, button labels, validation message wording, loading/empty/error states and accessibility checklist for the common routes.

How to implement alone: For each route write a small state table: loading, empty, valid, invalid, unauthorized and failed. Define exact field names/labels shared by web/mobile, then create fixtures matching OpenAPI. Check accessibility meanings even when layouts differ.

Deliver: docs/ui-contract-v1.md, mock JSON fixtures, React smoke tests and Flutter widget smoke tests. **Pass when:** Each route has a documented success, loading, empty and error state; wording is identical unless platform-specific.

Sprint 1 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S1-API-01

Build: Create FastAPI project factory, /api/v1/health/live, /api/v1/health/ready, configuration loader and dev/staging/prod settings validation.

How to implement alone: Separate app creation, configuration and mounted routers. Liveness shows the process is running; readiness checks required dependencies without leaking connection details. Validate required settings at startup. Test healthy, unavailable database and missing-setting cases.

Deliver: backend/app/factory.py, config.py, health router, pytest tests and .env.example without secrets. **Pass when:** Application fails clearly when required settings are absent; health tests return expected status and schema.

S1-API-02

Build: Define API v1 conventions: response envelope, ApiError schema, validation error format, pagination, correlation ID and OpenAPI examples; implement middleware.

How to implement alone: Choose stable error codes and safe messages, then document success/error examples and pagination. Add a correlation ID that passes through logs and responses. Generate TypeScript/Dart clients and compile both before screens depend on the schema.

Deliver: docs/api-standard-v1.md, Pydantic response/error models, correlation middleware and unit tests. **Pass when:** A sample endpoint returns correlation ID and standardized 200, 422 and 500 payloads; React/Flutter reviewers approve the schema.

S1-API-03

Build: Set up backend quality gates: Ruff/formatting, type checking, pytest coverage, secret scan and CI workflow; create auth dependency stub used by protected sample route.

How to implement alone: Add checks to the existing pipeline and prove a deliberately failing check blocks a candidate. An auth stub is for isolated tests only, never production identity. Keep the sample protected route and test fixtures easy to run from a clean checkout.

Deliver: CI config, requirements/pyproject updates, protected sample route, tests for missing/invalid mock token. **Pass when:** A deliberately failing lint/test blocks CI; successful run is attached to the sprint evidence.

Sprint 1 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S1-DAT-01

Build: Create Databricks catalog and schema design for dev/staging/prod, naming rules and ownership map; draft DDL for users, portfolios and system events.

How to implement alone: Design separate logical development/test/demo namespaces and a domain ownership map. Use Postgres operational tables for user-owned mutable records; Delta for events/history. Logical schemas do not provide the isolation of separate production environments.

Deliver: data/ddl/001_core.sql, docs/data-naming.md and catalog/schema creation notebook. **Pass when:** DDL is repeatable, uses explicit types, includes created_at/updated_at and passes peer review.

S1-DAT-02

Build: Implement tables for users, user_profiles, portfolios, portfolio_holdings and application_events with seed data notebook.

How to implement alone: Apply the source conflict rule: users, profiles, portfolios and holdings belong in operational Postgres with Alembic; application events/analytical copies may be Delta. Run initialization twice safely and keep synthetic seed records separate from real-data ingestion.

Deliver: Databricks notebook 01_create_core_tables, seed notebook and sample query output. **Pass when:** Notebooks can run twice without duplicate schema errors; seed rows can be queried through SQL Warehouse.

S1-DAT-03

Build: Create data quality checks for non-null IDs, valid timestamps, unique emails, positive holding quantity and portfolio ownership; publish a basic SQL monitoring query.

How to implement alone: Write checks as explicit rules with count and offending IDs, not screenshots of arbitrary queries. Inject one invalid test row per rule in isolated fixtures to show detection. Prevent invalid operational writes with constraints as well as downstream checks.

Deliver: notebooks/02_core_data_quality, expected- results table and screenshot/query evidence. **Pass when:** Checks identify intentionally bad rows and pass for clean seed data; failures include table, rule and offending count.

Sprint 1 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S1-AI-01

Build: Write the FinEx AI use-case and safety specification: supported intents, prohibited claims, answer template, disclaimers, data freshness and citation requirements.

How to implement alone: List supported educational intents and define what evidence each needs. Create at least 30 labelled prompts with allowed/refuse/insufficient-evidence outcomes. Scores and balances stay deterministic; the policy must forbid fabricated citations, prices and return guarantees.

Deliver: docs/ai-answer-policy-v1.md and a labeled CSV with at least 30 prompts. **Pass when:** Every prompt has expected intent, required sources, disclaimer and pass/fail criteria.

S1-AI-02

Build: Create a Databricks RAG experiment notebook that ingests three approved sample financial documents, chunks text, creates embeddings through the selected test model and stores retrievable records.

How to implement alone: Use three genuine approved public documents, with URL, period, checksum and page metadata. Inspect extracted text before embedding. Keep a replaceable retriever/model adapter; blocked cloud quota may use controlled fixtures for tests, never fake live product responses.

Deliver: notebooks/ai/01_rag_prototype, sample document registry and retrieval output for five questions. **Pass when:** Each retrieved chunk includes source, document date, section and chunk ID; notebook is reproducible.

S1-AI-03

Build: Build AI evaluation harness v0 with exact- match/keyword checks for education answers, citation presence, disclaimer presence, latency and manual rating fields.

How to implement alone: Create a repeatable runner that records question ID, versions, answer, required evidence, checks and manual rating. Separate citation presence from actual citation support. Track latency and failures rather than hiding unsupported cases in one average score.

Deliver: ai/evaluation/evaluate.py, evaluation CSV template and baseline report. **Pass when:** Script produces per-case results and aggregate scores; failed cases are traceable to prompt ID.

Sprint 2 / Identity, onboarding and risk profile

Implementation owner: Shivraj Dhote. **Source:** Section 6.2 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

A user signs in, records versioned consent/profile information and receives a deterministic, explainable risk profile on both clients.

Recommended solo dependency order

Freeze questionnaire/consent → identity/risk tables → deterministic scorer → auth/profile/risk APIs → explanation → React then Flutter → cross-client and security checks.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Auth/profile/consent/risk flow completes on both clients; approved deterministic cases pass; ownership and explanation checks pass.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 2 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S2-PRD-01

Build: Build React login, consent, onboarding and risk questionnaire screens using the shared UI contract; integrate auth/profile endpoints when available.

How to implement alone: Build entry, explicit consent and profile steps from the frozen contract. Preserve unfinished input where appropriate and show session expiry. Connect the real endpoints before acceptance; mock sign-in is not real authentication.

Deliver: React pages, typed services, validation tests and screenshots. **Pass when:** User can resume incomplete onboarding; required consent is explicit; API and UI validation messages match.

S2-PRD-02

Build: Build Flutter login, consent, onboarding and risk questionnaire flow with secure token storage and resume behavior.

How to implement alone: Implement the chosen OAuth mobile flow and supported secure token storage. Check cancel, expired session, background/resume and interrupted onboarding. Match web field semantics and consent version rather than copying desktop layout literally.

Deliver: Flutter feature modules, widget tests and emulator demo recording. **Pass when:** Fresh install, returning user and expired-token flows work; no token is logged.

S2-PRD-03

Build: Create cross-platform onboarding test automation and analytics events for start, step completion, validation failure, risk completion and abandonment.

How to implement alone: Automate new/returning user and invalid-field journeys on both clients. Analytics events should contain safe step/status IDs, not tokens, answers or personal fields. Distinguish an observed abandonment signal from an inferred user intention.

Deliver: React tests, Flutter integration tests, event specification and evidence from test event table. **Pass when:** Event payload contains no answers that reveal sensitive financial details; same logical event names are used on both platforms.

Sprint 2 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S2-API-01

Build: Implement /api/v1/auth/google/callback, JWT creation/validation, refresh/logout and /api/v1/users/me using the shared error envelope.

How to implement alone: Validate the provider identity server-side and bind it to the correct application user. Implement issuer/audience/expiry and the approved refresh/logout/revocation policy. Validate callback/state semantics for the chosen OAuth flow; do not accept arbitrary body user IDs.

Deliver: auth router/service/repository, token tests, OpenAPI examples and Postman/HTTP evidence. **Pass when:** Valid mock identity creates or loads a user; expired/revoked tokens return standardized 401; logout invalidates refresh token.

S2-API-02

Build: Implement onboarding and consent endpoints: GET/PUT /users/me/profile, POST /onboarding/complete, GET /risk/questions and POST /risk/assessments.

How to implement alone: Expose profile, consent, completion, question and assessment schemas consistently. Enforce required consent and questionnaire versions and ownership. Store server-controlled timestamps. Reject incomplete answers with field-specific errors rather than silently assigning a lower score.

Deliver: routers/onboarding.py, schemas, services, repository methods and tests. **Pass when:** Invalid answer IDs are rejected; questionnaire version is

S2-API-03

Build: Create contract and integration tests for authentication, profile, consent and risk endpoints; add fixtures for new, returning, expired-token and incomplete-profile users.

How to implement alone: Use two test users and expired/invalid credentials to exercise every protected operation. Test repeated save/completion and unsupported questionnaire versions. Check actual route registration and generated clients, not only pure functions.

Deliver: pytest integration suite, test data reset script and endpoint coverage report. **Pass when:** All critical paths pass; each validation/error code has at least one test; reset script restores clean state.

Sprint 2 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S2-DAT-01

Build: Create tables users, user_profiles, user_preferences, user_consents and auth_audit_events, including merge/upsert notebook.

How to implement alone: Place users/profiles/preferences/consent and session-related records in Postgres. Store approved audit copies/history separately. Preserve provider subject uniqueness and consent version/time; do not model the email address alone as an immutable provider identity.

Deliver: data/ddl/010_identity.sql and notebook 10_identity_upsert. **Pass when:** One user can have versioned consent records; email uniqueness and audit timestamps are enforced by checks.

S2-DAT-02

Build: Create versioned risk_questionnaires, risk_questions, risk_assessments and risk_assessment_answers tables plus a notebook to load questionnaire v1.

How to implement alone: Use stable question/answer IDs and immutable released questionnaire versions. An assessment references its answer set and scoring configuration. Keep old assessments interpretable after new questions are published; avoid editing old versions in place.

Deliver: DDL, questionnaire seed file and loader notebook. **Pass when:** Question order, scoring weight and version are queryable; historical assessments remain linked to original version.

S2-DAT-03

Build: Create identity/risk data quality checks and an audit query showing profile completeness, consent status, duplicate identity and orphan risk answers.

How to implement alone: Check profile completeness against required fields, not a hardcoded true flag. Detect duplicate subjects, missing consent and orphan answers. Audit queries must respect access limits and avoid exporting unnecessary personal data.

Deliver: notebook 12_identity_risk_dq and SQL dashboard query. **Pass when:** Injected duplicate/orphan rows are detected; clean test dataset returns zero critical failures.

Sprint 2 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S2-AI-01

Build: Implement deterministic risk scoring v1 from approved questionnaire weights, separating willingness and capacity scores and mapping to conservative/moderate/aggressive.

How to implement alone: Use the approved willingness and capacity weights and band boundaries. Implement a pure scorer and configuration version; test at least 12 profiles including boundaries. Missing approvals are a specification blocker, not a reason to invent risk thresholds.

Deliver: ai/risk/scoring.py, configuration file and tests for at least 12 profiles. **Pass when:** Same answers always produce same score; boundary cases are documented; no LLM is used for numeric scoring.

S2-AI-02

Build: Create prompt/template that explains the deterministic risk result in beginner language, cites influencing answers and includes limitations.

How to implement alone: Feed the fixed score, components and influencing answers to the explanation layer. Validate that every numeric claim matches those inputs. Provide a deterministic text template when the model is unavailable or fails validation.

Deliver: prompt file, response schema and 10 generated explanation examples. **Pass when:** Explanation never changes numeric score, mentions top factors and does not promise returns.

S2-AI-03

Build: Extend evaluation harness with risk-profile cases: score consistency, factor coverage, prohibited claims, readability and disclaimer checks.

How to implement alone: Report numeric score accuracy separately from explanation readability, factor coverage and safety. Include missing answers, extreme/boundary profiles and prohibited promises. Passing a score test does not automatically validate the generated explanation.

Deliver: risk evaluation dataset, automated checks and baseline report. **Pass when:** All deterministic scores match expected values; explanation safety pass rate is reported separately.

Sprint 3 / Market data, watchlist and portfolio

Implementation owner: Shivraj Dhote. **Source:** Section 6.3 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

A user searches actual supported instruments, maintains watchlists and sees persistent holdings with sourced and reproducible valuation.

Recommended solo dependency order

Freeze instrument/quote schema → real ingestion and history → portfolio calculation/API → web then mobile → sourced comparison/explanations → DQ and parity.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Freshness/source are visible; watchlist/portfolio reconcile with stored data; no missing quote becomes zero.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 3 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S3-PRD-01

Build: Build React market search, watchlist and portfolio dashboard: quote freshness badge, holdings table, allocation chart, loading/empty/error states.

How to implement alone: Show source, observation time and freshness beside market values. Build search, persisted watchlist and holdings summary using backend metrics. Charts need empty/missing-data treatment and text alternatives; do not convert absent prices to zero.

Deliver: React feature pages, services, tests and accessibility screenshots. **Pass when:** User can add/remove symbol, view freshness, and see portfolio totals without client-side financial calculations.

S3-PRD-02

Build: Build Flutter market search, watchlist and portfolio dashboard with the same fields and states as React.

How to implement alone: Retain source/time/currency and error meanings on small screens. Test search, add/remove, refresh and offline/provider-failure states. Compare a single test portfolio across web/mobile instead of building a second valuation formula.

Deliver: Flutter feature modules, widget/integration tests and Android demo. **Pass when:** Cross-platform field and validation parity checklist passes.

S3-PRD-03

Build: Implement accessible chart wrappers, API response mapping tests and end-to- end watchlist/portfolio flows for React and Flutter.

How to implement alone: Use known quote/holding fixtures to compare chart labels and totals against the response. Test keyboard/semantics and unreadable narrow-screen layouts. Real-data verification additionally compares displayed values with saved source records.

Deliver: Chart components, test suites and data reconciliation evidence. **Pass when:** Chart labels match API values; watchlist change persists after refresh; portfolio UI equals API snapshot.

Sprint 3 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S3-API-01

Build: Implement /market/search, /market/quotes/{symbol}, /watchlists and watchlist item CRUD with cache/freshness fields.

How to implement alone: Normalize provider symbols into canonical instrument IDs while retaining the provider symbol. Persist caller-owned watchlist entries with duplicate protection. Return documented unsupported/stale/unavailable outcomes and separate observation time from retrieval time.

Deliver: routers/market.py, watchlists.py, service/repository code and tests. **Pass when:** Unsupported symbol, duplicate watchlist item, stale provider and provider failure return documented outcomes.

S3-API-02

Build: Implement portfolio CRUD, holdings validation, snapshot calculation and /portfolios/{id}/summary with deterministic totals and allocations.

How to implement alone: Implement holdings validation and decimal calculations with clear cost basis, fees and currency conventions. Report partial valuation when quotes are missing. Enforce ownership in repositories/services, not only hidden client buttons.

Deliver: portfolio router/service, calculation module and unit/integration tests. **Pass when:** Allocation and gain/loss calculations pass fixed expected cases; user cannot access another user's portfolio.

S3-API-03

Build: Add caching, pagination, provider error mapping and contract tests for market/watchlist/portfolio endpoints.

How to implement alone: Cache only suitable reads and record expiry/status. Bound retries and map upstream errors to safe stable codes. Pagination must preserve deterministic ordering. Test quota/outage and old-cache cases without mislabelling cached values as fresh.

Deliver: test suite, cache configuration and API contract report. **Pass when:** Safe GET calls use cache; stale data is labeled; failures never masquerade as zero price.

Sprint 3 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S3-DAT-01

Build: Create securities, market_quotes, market_price_history, watchlists and watchlist_items tables with provider/freshness metadata.

How to implement alone: Watchlists/holdings stay operational; history and portfolio snapshots belong in Delta. Include provider, observed_at, ingested_at, currency and adjustment basis. Use unique natural keys for history and stable identifiers for instruments.

Deliver: data/ddl/020_market.sql and table creation notebook. **Pass when:** Each quote records source, observed_at, ingested_at and freshness class; watchlist uniqueness is testable.

S3-DAT-02

Build: Build market-data ingestion notebook/job: symbol normalization, provider adapter, retry, deduplication and MERGE into quote/history tables.

How to implement alone: Import genuine provider responses or permitted downloaded reports. Preserve source files/hashes, normalize, validate, quarantine bad rows and merge idempotently. Do not scrape around access controls. Re-running the same observation must not duplicate history.

Deliver: notebooks/market/20_ingest_quotes, provider adapter and run log table. **Pass when:** Repeated input does not duplicate history; invalid symbols are quarantined; freshness label is calculated.

S3-DAT-03

Build: Create market freshness and portfolio data quality checks: stale quote, duplicate quote, orphan watchlist item, invalid holding quantity and allocation sum.

How to implement alone: Define warning versus blocking quality checks. Distinguish market holidays from missing feeds. Check orphan watchlists, impossible quantities and allocation sums only where full valuation exists; incomplete valuation must be visible.

Deliver: DQ notebook, alert thresholds and sample failed-run report. **Pass when:** Checks distinguish warning versus blocking failure and identify affected symbols/users.

Sprint 3 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S3-AI-01

Build: Create structured company/fund comparison prompt and response schema for returns, fees, risk, data date, assumptions and source list.

How to implement alone: Use only available sourced company/fund fields. A price-only dataset cannot support fund fees or dividend facts. Every comparison metric needs its source/date/basis, otherwise return unavailable; do not let the model fill gaps.

Deliver: prompt templates, JSON schema and 12 comparison examples. **Pass when:** Output does not invent missing metrics; every metric includes date/source or 'not available'.

S3-AI-02

Build: Implement portfolio explanation tool that receives deterministic backend metrics and produces beginner-friendly concentration/diversification narratives.

How to implement alone: Pass a small deterministic metric object to the model or template. Explain concentration/diversification without new arithmetic or personalised trade instructions. Link narratives to the snapshot version so later prices do not silently change their meaning.

Deliver: tool interface, prompt, model call wrapper and sample outputs. **Pass when:** No arithmetic is delegated to the LLM; narrative numbers exactly match tool input.

S3-AI-03

Build: Build factuality/evaluation set for comparisons and portfolio explanations, including missing-data, stale-data and contradictory-source cases.

How to implement alone: Build at least 20 cases covering supported, missing, stale and contradictory inputs. Verify numbers against the service result and sources against actual evidence. Report factuality, citations, freshness and prohibited-claim outcomes separately.

Deliver: evaluation dataset and automated result report. **Pass when:** Report separates citation, numeric consistency, freshness disclosure and prohibited-claim scores.

Sprint 4 / AI financial assistant and RAG

Implementation owner: Shivraj Dhote. **Source:** Section 6.4 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

A user asks educational/research questions and receives supported answers with inspectable citations, history and feedback.

Recommended solo dependency order

Register documents/chats → ingestion/chunks/index → retrieval evaluation → orchestration/guardrails → chat APIs → React then Flutter → feedback/tracing and UAT.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Answers cite approved supporting documents; conversation ownership and feedback work; retrieval and safety results are recorded.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 4 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S4-PRD-01

Build: Build React AI chat: conversation list, composer, response states, citation cards, copy, feedback and delete confirmation.

How to implement alone: Build conversation list/composer and distinct pending, successful, insufficient-evidence and error states. Render citations as inspectable source cards. Confirm deletion before action; show retained-data policy accurately. Safely render model content without executable markup.

Deliver: React chat feature, typed service, unit/E2E tests and accessibility report. **Pass when:** Retry preserves prompt; citation opens metadata; duplicate send is blocked; errors do not erase history.

S4-PRD-02

Build: Build Flutter AI chat with history, citations, feedback, session expiry and safe rendering of long financial content.

How to implement alone: Handle long answers, keyboard, scroll and expired sessions on the supported device. Keep citations visible and feedback accessible. Reopen history using the same authenticated owner and server pagination rules as web.

Deliver: Flutter chat module, widget/integration tests and emulator evidence. **Pass when:** State survives rebuild; long content does not overflow; citation fields match React.

S4-PRD-03

Build: Create cross-platform chat test suite and analytics: send, success, failure, citation click, feedback, delete and session expiry.

How to implement alone: Test citation open, failure/retry, feedback and deletion on both clients. Record safe event names and IDs, never raw prompts/tokens by default. Cross-client parity means equal data/status meaning, not identical pixel layout.

Deliver: React and Flutter automated tests, event dictionary update and sample event queries. **Pass when:** No prompt content or token is included in analytics; all critical chat states are tested.

Sprint 4 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S4-API-01

Build: Implement conversation/session/message/feedback endpoints with user isolation and pagination.

How to implement alone: Use owner-scoped conversation/message/feedback repositories and stable pagination. Validate parent conversation ownership for every message action. Decide delete/retention semantics and return a clear outcome; a guessed conversation ID never grants access.

Deliver: chat router/service/repository, OpenAPI examples and tests. **Pass when:** Users can only access their own history; delete behavior is defined; pagination is deterministic.

S4-API-02

Build: Implement AI orchestration service: classify intent, retrieve context, call model serving, apply timeout/retry policy, invoke guardrails and return citation-rich schema.

How to implement alone: Classify supported intent, retrieve only permitted context, invoke the model through an adapter and validate output/citations. Bound timeout and retry. Treat retrieved text as untrusted evidence; it cannot change authorization or invoke arbitrary tools.

Deliver: orchestration service, client adapters, unit/integration tests and tracing fields. **Pass when:** Timeout returns safe error; each answer has model/version, data date and citations; no silent fallback to fabricated response.

S4-API-03

Build: Implement feedback endpoint, correlation tracing across API/retrieval/model calls and integration tests for education, comparison and refusal intents.

How to implement alone: Connect API/retrieval/model spans using safe correlation IDs. Make duplicate feedback behaviour explicit. Test supported education/comparison and refusal/insufficient-evidence outcomes, including slow or malformed upstream output.

Deliver: feedback API, trace tests and contract evidence. **Pass when:** One correlation ID links message, retrieval event and model call; feedback references the exact answer version.

Sprint 4 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S4-DAT-01

Build: Create document_registry, document_chunks, ai_conversations, ai_messages, ai_feedback and retrieval_events tables.

How to implement alone: Conversation/message/feedback metadata is operational Postgres. Registry/chunks/retrieval/evaluation history use the agreed analytical/document layer. Keep ownership/retention metadata and source lineage; do not move the mutable chat store to Delta simply because an older card says Delta.

Deliver: data/ddl/030_ai_rag.sql and creation notebook. **Pass when:** Tables support document versioning, user isolation, citations, latency and model version fields.

S4-DAT-02

Build: Create and populate the Databricks Vector Search index and retrieval workflow with metadata filters for company, document type, period and date.

How to implement alone: Index only validated chunks and preserve company/type/reporting-period/date filters. Prove the selected account actually supports the intended index service. An alternative retriever must be recorded as a deviation; it is not evidence of a completed Vector Search deployment.

Deliver: index creation workflow, query helper and retrieval evidence. **Pass when:** Top-k results respect filters; deleted/superseded documents are excluded.

S4-DAT-03

Build: Create AI data lineage and quality checks for missing metadata, duplicate chunks, orphan citations, failed embeddings and unlinked feedback.

How to implement alone: Check source registry references, duplicate hashes, missing pages, failed embeddings and citations to non-existent chunks. Track document replacement so answers retain their original source version. Respect deletion/retention requirements for private material.

Deliver: DQ notebook, lineage query and failed-run report. **Pass when:** All synthetic failures are detected; report identifies pipeline run and affected document/message.

Sprint 4 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S4-AI-01

Build: Build document ingestion pipeline for approved filings/factsheets: registry entry, text extraction, section-aware chunking, metadata validation and embedding job.

How to implement alone: Register approved real filings/factsheets; extract and inspect pages/tables; chunk by sections and validate metadata before embedding. Quarantine unreadable scans instead of indexing empty text. Version extraction/chunk/model settings for reproducibility.

Deliver: production-style Databricks workflow, reusable Python modules and run evidence. **Pass when:** Every chunk has source/date/section/page reference; invalid documents are quarantined with reason.

S4-AI-02

Build: Implement retrieval evaluation notebook measuring recall@k, metadata filter correctness and stale-document exclusion on a labeled query set.

How to implement alone: Create at least 40 representative questions with expected supporting document/chunk labels and a rubric. Measure recall@k and filter correctness with a defined denominator. Test obsolete reporting periods explicitly and record actual outcomes.

Deliver: evaluation notebook, dataset and baseline metrics. **Pass when:** Each query records expected source, retrieved sources and failure reason; metrics are reproducible.

S4-AI-03

Build: Implement answer guardrails and citation validator: block unsupported numeric claims, require citations for document claims, add freshness/limitations/disclaimer.

How to implement alone: Validate each returned citation against retrieved IDs and check supported numeric claims. Missing evidence produces a safe fallback. Red-team malicious source instructions, invented citations and requests for another user data before accepting generated answers.

Deliver: guardrail module, tests and red- team cases. **Pass when:** Answers failing citation/numeric checks are rejected or rewritten into a safe fallback; reasons are logged.

Sprint 5 / Paper trading

Implementation owner: Shivraj Dhote. **Source:** Section 6.5 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

A user practices supported simulated orders whose cash, trades and positions reconcile exactly and remain separate from real-money execution.

Recommended solo dependency order

Freeze fill/fee/rounding policy → operational ledger constraints → atomic service → concurrency/replay tests → snapshots/reconciliation → both clients → coaching evaluation.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Account/order/trade/cash/position operations are atomic, idempotent and reconcilable; all visible trading is clearly simulated.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 5 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S5-PRD-01

Build: Build React paper-trading screens: virtual account summary, market buy/sell order ticket, confirmation, positions, order history and visible simulation disclaimer.

How to implement alone: Show virtual cash and simulation labels at every order decision. Keep the same logical request identity during retries; show pending/unknown/rejected distinctly. Read confirmed positions from server rather than optimistic financial arithmetic.

Deliver: React feature module, service layer, form tests and E2E buy/sell tests. **Pass when:** Invalid quantity/cash/holding cases cannot submit; duplicate clicks create one request; all screens label prices as live/delayed/latest.

S5-PRD-02

Build: Build Flutter paper-trading account, order ticket, positions and history with the same validation and disclaimer as React.

How to implement alone: Handle double tap, interrupted network and app resume without creating a new logical order. Display price source/time/fill assumptions. A timeout means outcome unknown until safe replay/lookup confirms it.

Deliver: Flutter feature module, widget/integration tests and emulator demo. **Pass when:** Cross-platform validation matrix passes; app recovers cleanly from price-unavailable response.

S5-PRD-03

Build: Create paper-trading E2E and reconciliation tests covering account creation, buy, partial sell, insufficient cash, excessive sell, stale price, duplicate request and reset.

How to implement alone: Build a fixed starting account and expected results for buy, partial sell, insufficient funds, over-sell, stale price, replay and reset. Check both screen state and actual ledger; a success toast alone is not execution proof.

Deliver: React/Flutter E2E suite, reconciliation sheet and defect report. **Pass when:** UI balances, positions and backend ledger agree after every scenario; reset requires explicit confirmation.

Sprint 5 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S5-API-01

Build: Implement paper account creation/reset, order submit/list/get and positions endpoints with idempotency key and user isolation.

How to implement alone: Scope account/order routes to the owner and persist idempotency keys with request fingerprints. Same key/payload returns the original result; different payload conflicts. Reset must be an explicit controlled operation with audit/reconciliation semantics.

Deliver: paper_trading router/service/repository and tests. **Pass when:** Unauthorized access is blocked; repeated idempotency key returns same result; reset behavior is audited.

S5-API-02

Build: Implement deterministic execution/ledger service for market buy/sell: validate usable price, cash/quantity, create trade, cash ledger entry and updated position atomically.

How to implement alone: Within one supported database transaction validate available cash/quantity, write order/trade/ledger and update position/cash. Use locking or equivalent isolation for concurrent requests. Freeze fees, rounding, quantity and fill policy before coding.

Deliver: execution service, transaction boundary design and unit tests. **Pass when:** Expected cash, average cost, realized and unrealized values match test vectors; failed order changes no ledger state.

S5-API-03

Build: Add security, concurrency and failure tests for paper trading, including double submit, simultaneous orders, missing quote and rollback.

How to implement alone: Exercise simultaneous orders against the same balance, conflicting replay, missing quote and injected mid-transaction failure. No partial write or duplicate fill may survive. A real brokerage execution API is outside this product.

Deliver: integration/load tests and defect report. **Pass when:** No negative cash/position from tested concurrency; partial writes are absent after forced failure.

Sprint 5 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S5-DAT-01

Build: Create paper_accounts, paper_orders, paper_trades, paper_positions, paper_cash_ledger and paper_portfolio_snapshots tables.

How to implement alone: Orders/trades/account/cash ledger/positions belong in transactional Postgres; analytical snapshots belong in Delta. Add uniqueness and ownership constraints plus request fingerprint/result records. Preserve ledger correction history instead of destructive edits.

Deliver: data/ddl/040_paper_trading.sql and table notebook. **Pass when:** Order, trade and ledger identifiers are traceable; cash ledger is append-only; reset is represented without deleting audit history.

S5-DAT-02

Build: Build snapshot workflow and deterministic SQL/Python calculations for account equity, cost basis, realized/unrealized P&L and daily return.

How to implement alone: Calculate equity/cost basis/realized and unrealized P&L; from verified ledger/trades/prices. Define dividend/fee/corporate-action handling and daily-return denominator. If support is absent label the metric unavailable or limited, not fully accurate investment performance.

Deliver: Databricks workflow, calculation module and expected- results notebook. **Pass when:** Results match hand- calculated test portfolios; missing price is flagged, not treated as zero.

S5-DAT-03

Build: Create reconciliation notebook comparing orders, trades, positions, cash ledger and snapshots; produce break report.

How to implement alone: Reconcile cash and quantities across account, trade, ledger and snapshot. Produce a break report with exact affected IDs, expected and actual amounts. Test both a clean scenario and an intentionally broken isolated fixture.

Deliver: reconciliation notebook, break table and daily check query. **Pass when:** Injected duplicate/missing ledger entries are identified; clean scenario balances to zero difference.

Sprint 5 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S5-AI-01

Build: Create paper-trading coach prompt that explains order outcome, portfolio impact, risk and learning points without encouraging real-money execution.

How to implement alone: Explain a confirmed simulated order outcome and its deterministic impact. Never suggest that a virtual result guarantees real returns. Reject unsupported advice requests and preserve all price/date/fee assumptions from the tool result.

Deliver: prompt, response schema and 15 scenario outputs. **Pass when:** Narrative matches trade values, states simulation status and includes risk/learning explanation.

S5-AI-02

Build: Build performance narrative pipeline using deterministic paper account metrics to summarize contributors, concentration and behavior patterns.

How to implement alone: Use the frozen performance snapshot for narratives and list actual contributing positions. Do not generate weekly performance from missing history. Record prompt/model/snapshot versions and provide a non-LLM summary fallback.

Deliver: AI tool interface, prompt and sample weekly summaries. **Pass when:** No new numeric calculation by LLM; every stated amount/percentage maps to input field.

S5-AI-03

Build: Create safety and factuality evaluation for paper-trading coaching: no guaranteed return, no real-trade instruction, exact numeric consistency and disclaimer.

How to implement alone: Compare every output amount with tool input and check simulation/disclaimer language. Include losses, missing quotes, concentrated accounts and requests to execute real money. Fail unsafe/fabricated output rather than hiding it behind generic warnings.

Deliver: evaluation set, automated tests and baseline report. **Pass when:** All numeric mismatches are detected; unsafe prompts produce educational/refusal response.

Sprint 6 / Smart alerts and conversational portfolio intelligence

Implementation owner: Shivraj Dhote. **Source:** Section 6.6 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

A user manages controlled alerts and asks portfolio questions answered from deterministic, authorized metrics and explicit scenarios.

Recommended solo dependency order

Freeze alert/metric definitions → stores/snapshots → evaluator/delivery → portfolio tools → both client views → intent/explanation evaluation. Revisit goal-off-track after S7.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Alerts avoid duplicate spam; read state persists on both clients; portfolio/scenario answers reproduce deterministic metrics.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 6 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S6-PRD-01

Build: Build React alert management, notification center and portfolio conversation views with rule creation, unread counts and scenario cards.

How to implement alone: Build rule create/edit/disable, notification list/unread state and portfolio question views. Explain that triggers depend on data freshness and delivery is best effort. Do not represent a delivered message as a trade.

Deliver: React features, service tests and E2E flow. **Pass when:** User can create/deactivate alert, mark notifications read and ask portfolio question with displayed data date.

S6-PRD-02

Build: Build Flutter alert management, notification center and conversational portfolio views matching React fields and safety copy.

How to implement alone: Use the same rule fields/read states and scenario assumptions as web. Register notification tokens only through authenticated supported flows and protect them. Keep in-app notifications usable even when push is denied or fails.

Deliver: Flutter modules, tests and emulator demo. **Pass when:** Push/local-notification foundation is isolated; in-app notification state persists after refresh.

S6-PRD-03

Build: Create cross-platform E2E tests for alert creation/trigger/read and portfolio questions; verify displayed numbers against API response.

How to implement alone: Test create-trigger-read on both clients and verify metric values against API output. Exercise cooldown, duplicate evaluation and denied notification permission. Confirm read state persists across devices.

Deliver: E2E suite and reconciliation report. **Pass when:** All displayed amounts/percentages match API; stale data and unsupported questions show correct limitation.

Sprint 6 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S6-API-01

Build: Implement alert rule CRUD and evaluation endpoints for price, percentage movement, concentration, risk change and goal off-track conditions.

How to implement alone: Implement price, movement, concentration and risk-change conditions from versioned rules. Goal-off-track stays disabled until S7 goal contracts/values exist. Enforce cooldown, deduplication, ownership and deterministic evaluation; do not fabricate a goal value to unblock S6.

Deliver: alerts router/service/evaluator and tests. **Pass when:** Each rule has explicit units, comparison operator, active status and cooldown; invalid thresholds are rejected.

S6-API-02

Build: Implement deterministic portfolio intelligence service returning allocation, concentration, diversification, top contributors/detractors, volatility input and scenario results.

How to implement alone: Return defined allocation, concentration, diversification, contributor and scenario metrics with source/snapshot dates. Volatility needs a valid return history and method. Missing inputs must remain unavailable rather than guessed by the assistant.

Deliver: portfolio_intelligence service, response schemas and calculation tests. **Pass when:** All numeric outputs are deterministic and versioned; missing data produces explicit unavailable fields.

S6-API-03

Build: Implement notifications list/read/mark-all- read endpoints and integration tests linking alert event to notification delivery status.

How to implement alone: Persist the in-app record and delivery attempts separately. Mark read/list operations are owner-scoped; a failed FCM attempt must not delete the in-app event. Define retry policy and unique event-to-notification mapping.

Deliver: notifications router/service and contract tests. **Pass when:** Unread counts remain consistent; user cannot read another user's notifications; failed delivery remains auditable.

Sprint 6 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S6-DAT-01

Build: Create alert_rules, alert_events, notifications and notification_delivery_attempts tables with indexes/partition strategy and status history.

How to implement alone: Store rules, events, notifications and delivery attempts operationally with status and dedupe keys. Index owner/read status and due-rule fields. Retain enough history to explain why a rule fired without retaining unnecessary personal content.

Deliver: data/ddl/050_alerts.sql and creation notebook. **Pass when:** One rule can create many events; delivery attempts are append-only; read status is queryable.

S6-DAT-02

Build: Build portfolio snapshot/metrics pipeline for allocation, sector exposure, concentration score, diversification score and change from prior snapshot.

How to implement alone: Generate versioned metrics/snapshots from verified holdings and market input. Sector data needs an approved source; missing sectors cannot be invented. Record previous/current snapshot comparison and incomplete-data status.

Deliver: Databricks workflow, metrics table and test notebook. **Pass when:** Metrics are reproducible and linked to snapshot timestamp; totals reconcile with portfolio summary.

S6-DAT-03

Build: Build alert evaluation job and monitoring queries for due rules, triggered events, duplicate suppression and delivery failures.

How to implement alone: Evaluate due rules with a bounded schedule and concurrency-safe claiming/deduplication. Log processed/triggered/suppressed/failed counts and last successful run. Verify schedule timezone and model actual EOD data availability.

Deliver: scheduled job/notebook, run log and monitoring SQL. **Pass when:** Same event is not repeatedly emitted within cooldown; failed rows are retryable and visible.

Sprint 6 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S6-AI-01

Build: Build portfolio-question intent router for holdings, allocation, concentration, performance, risk, scenario and unsupported requests.

How to implement alone: Route questions into a small documented intent set before choosing a tool. Unknown requests go to explicit fallback. Evaluate labelled examples including paraphrases and requests attempting to access another account.

Deliver: intent router, dataset and accuracy report. **Pass when:** Test accuracy meets agreed threshold; unsupported questions route to education or safe limitation.

S6-AI-02

Build: Build conversational portfolio explanation and optimization prompt/tool chain using deterministic metrics, user risk profile and educational recommendation constraints.

How to implement alone: Combine authorized deterministic metrics with risk profile and explicit scenario assumptions. Present educational factors, not guaranteed optimal trades. The model explains tool output; it does not modify holdings or execute orders.

Deliver: tool schemas, prompts and scenario examples. **Pass when:** Recommendations explain current versus suggested allocation, assumptions and risk; no automatic trade action is offered.

S6-AI-03

Build: Create scenario/risk evaluation set for concentration, 15% sector decline, missing price, conservative user and conflicting goal/risk cases.

How to implement alone: Test a hypothetical 15% sector decline with known exposure arithmetic, plus missing sector/quote, conservative profile and goal conflicts. Scenario output must be labelled hypothetical and reproduce tool values, never framed as a forecast.

Deliver: evaluation dataset and regression report. **Pass when:** Numeric consistency, profile alignment, disclaimer and recommendation transparency are scored separately.

Sprint 7 / Goals and spending trend analysis

Implementation owner: Shivraj Dhote. **Source:** Section 6.7 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

A user creates financial goals and reviews privacy-controlled spending imports, category corrections and evidence-based trends.

Recommended solo dependency order

Freeze goal/CSV/privacy contracts → operational data → deterministic projections/parser → category pipeline → preview/commit APIs → both clients → totals/privacy/AI evaluation.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Projections expose assumptions; CSV preview/commit protects descriptions and reconciles totals; categorization supports confidence/review/correction.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 7 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S7-PRD-01

Build: Build React goal creation/projection/progress and spending CSV upload/dashboard screens with assumption, privacy, loading and correction states.

How to implement alone: Build goal inputs/projection/progress and upload preview/commit/category correction. Show formula assumptions and masked descriptions. Validation errors must be visible before commit; incomplete imports cannot silently appear as complete spending history.

Deliver: React feature modules, services, tests and screenshots. **Pass when:** User can create/edit goal, compare scenarios, upload CSV, correct categories and view monthly trends.

S7-PRD-02

Build: Build Flutter goal and spending screens with file picker abstraction, progress charts and category correction flow.

How to implement alone: Use a supported file-picker abstraction and the same server preview/commit logic. Test cancellation, large/invalid files and mobile correction. Keep chart text summaries and privacy wording equivalent to web.

Deliver: Flutter feature modules, tests and emulator evidence. **Pass when:** Same validation/assumption/privacy wording as React; file

S7-PRD-03

Build: Create cross-platform upload/projection tests, accessible chart text summaries and analytics events for goal/spending flows.

How to implement alone: Test projection cases and CSV reconciliation through both clients. Events contain safe status/count fields, not raw descriptions. Accessible text summaries must agree with chart values and date ranges.

Deliver: React/Flutter tests, chart summaries and event evidence. **Pass when:** No raw transaction description appears in analytics; displayed totals reconcile with API summaries.

Sprint 7 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S7-API-01

Build: Implement goal CRUD, projection and progress endpoints using deterministic projection module with explicit assumptions.

How to implement alone: Define rate conversion, contribution timing, duration, rounding and progress semantics. Return formula version and assumptions with results. Implement zero-rate cases and validation explicitly; no LLM calculates the projection.

Deliver: goals router/service/repository, calculation module and tests. **Pass when:** Invalid dates/amounts are rejected; projection includes assumptions and gap; ownership enforced.

S7-API-02

Build: Implement transaction CSV upload, validation, masked storage, category review/update and monthly summary endpoints.

How to implement alone: Bound upload size/rows/types, parse to a temporary scoped preview and commit only the reviewed accepted rows under the specified policy. Implement duplicate detection, masking and user correction. Guard exported spreadsheet cells against formula injection where applicable.

Deliver: spending router/service, upload parser and tests. **Pass when:** Malformed files return row- level errors; duplicate upload is identified; raw descriptions are protected.

S7-API-03

Build: Create privacy, authorization and integration tests for goals/spending; verify file size/type limits, masking, user isolation and summary reconciliation.

How to implement alone: Use two users and malformed, oversized, duplicate and mixed-validity files. Verify summary totals equal committed rows and failed commit leaves no unexplained partial state. Test masking and retention for raw uploads.

Deliver: integration/security tests and privacy verification report. **Pass when:** Cross-user access fails; disallowed files are rejected; response/logs do not expose raw sensitive content.

Sprint 7 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S7-DAT-01

Build: Create financial_goals, goal_contributions, goal_projections and goal_progress_snapshots tables with versioned projection assumptions.

How to implement alone: Keep goal inputs/contributions and current state operational; projection/progress history can be analytical snapshots. Store assumption/formula versions and ownership. Do not overwrite past projections so they appear calculated with today assumptions.

Deliver: data/ddl/060_goals.sql and creation notebook. **Pass when:** Projection assumptions are stored with each result; contribution history is append-only; goal ownership is explicit.

S7-DAT-02

Build: Create transaction_uploads, transactions, transaction_categories, monthly_spending_summaries and spending_insights tables; build CSV normalization and masking pipeline.

How to implement alone: Keep uploads/transactions/categories operational and monthly summaries/insights in analytics. Preserve import hash, row identity and correction provenance. Mask descriptions before optional external model calls; never ask other people for bank records.

Deliver: data/ddl/061_spending.sql, ingestion notebook and quarantine table. **Pass when:** Duplicate files/rows are detected; raw descriptions are masked for AI use; invalid rows are quarantined with reason.

S7-DAT-03

Build: Build goal/spending data quality and analytics checks: target/date validity, contribution reconciliation, category totals, duplicate transactions and monthly summary reconciliation.

How to implement alone: Check target/date consistency, contribution totals, duplicate transaction keys and category sums. Define handling of transfers/refunds/income to avoid treating every amount as spending. Reconcile by currency and accepted period.

Deliver: DQ notebook, monthly summary query and failure report. **Pass when:** Monthly category totals equal transaction totals; injected duplicates/invalid goals are detected.

Sprint 7 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S7-AI-01

Build: Create goal-planning explanation tool that interprets deterministic projection, required monthly contribution, shortfall and assumptions in simple language.

How to implement alone: Use deterministic projection/required-contribution/shortfall output to explain a goal in simple language. Include at least 12 goal cases as in the source evidence expectation. State assumed return rather than promising the target is achievable.

Deliver: tool schema, prompt and 12 scenario outputs. **Pass when:** Narrative does not guarantee target achievement; all amounts and dates match input.

S7-AI-02

Build: Build transaction categorization v1 using rules plus an ML/LLM fallback on masked descriptions; return category and confidence.

How to implement alone: Start with auditable categorization rules, then an approved ML/LLM fallback on masked descriptions. Return category, reason/confidence and review flag; user correction overrides a suggestion. A raw model confidence is not automatically calibrated probability.

Deliver: categorization module/notebook, labeled dataset and evaluation report. **Pass when:** Rules are deterministic; low- confidence items are marked for user review; raw sensitive descriptions are not sent externally.

S7-AI-03

Build: Build spending trend and anomaly evaluation: recurring expense, month-over- month increase, unusual transaction and savings-rate explanation cases.

How to implement alone: Build labelled recurring/month-over-month/unusual-transaction/savings-rate cases. Compare against deterministic aggregates and distinguish anomalies from proven fraud. Evaluate false positives and missing prior-month data; show uncertainty rather than invented comparisons.

Deliver: evaluation dataset, scoring script extension and report. **Pass when:** False-positive/false-negative examples are listed; insights use masked/category-level data.

Sprint 8 / Production hardening and release

Implementation owner: Shivraj Dhote. **Source:** Section 6.8 of the supplied Combined Handbook. All twelve source task IDs are preserved on the next four pages.

What this sprint produces

The complete accepted scope is secure, measurable, reproducible, deployable and ready for a documented web/mobile demonstration.

Recommended solo dependency order

Freeze candidate → security/data permissions → real deployment/builds/jobs → full regression/red-team/load → UAT fixes → recovery rehearsal → release evidence and handover.

Before you start

Confirm the previous accepted feature foundation, available contracts/examples and unresolved rules. Test-only scaffolds may use labelled mocks; production integration requires actual available data/services. For S1 use Week 0 readiness. For later sprints verify the exact predecessors on the source cards rather than assuming the calendar unblocks them.

A working session sequence

• Read the contract and write an independent expected result before coding.

• Complete data/service behaviour and test it before binding both clients.

• Integrate React, then Flutter with the same persona/input and compare outputs.

• Run DQ/evaluation/security checks for the affected feature and record evidence.

Exit gate

Critical UAT/security/data/AI checks pass; release artifacts/rollback are reproducible; every unresolved exception has an accepted disposition.

Full scope versus the demonstration

Every task below remains part of full completion. A deadline-driven subset may be demonstrated only with its limitations documented. Unimplemented AI, mobile or analytics work stays open; a mock or local replacement is not silently accepted as the specified service.

Your sprint handover to yourself

Write the accepted commit/build, contract/schema/prompt/source versions, verified run commands, fixtures/data provenance, results, limitations and next unblocked IDs. This makes the next sprint and any later mentor review reproducible even without teammates.

Sprint 8 / Product and client experience

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S8-PRD-01

Build: Harden React application: accessibility, route-level loading, bundle size, error monitoring, production environment, regression and deployment runbook.

How to implement alone: Run route/accessibility/performance checks against production configuration and generated client version. Deploy React using the approved pipeline; test direct refresh and error monitoring. Record bundle metrics before optimizing.

Deliver: release PR, accessibility/performance reports, production build and runbook. **Pass when:** Zero release-blocking accessibility defect; no localhost/secrets in bundle; rollback steps are tested.

S8-PRD-02

Build: Harden Flutter application: accessibility, performance, secure storage review, Android release build, iOS archive configuration, crash monitoring and store packet.

How to implement alone: Build a signed/authorized Android release through the project process and verify installation. Review secure storage, permissions and crash metadata. Prepare iOS configuration/source evidence where tooling exists; do not claim an archive built without macOS signing/tooling. Public store publishing remains outside scope.

Deliver: release build artifacts, test report, store metadata draft and runbook. **Pass when:** Android build installs and completes smoke flow; iOS configuration is reproducible; no secrets/logged tokens.

S8-PRD-03

Build: Coordinate cross-platform UAT fixes, verify shared copy/validation/data values and produce final user help/known-limitations content.

How to implement alone: Run final same-persona journeys on both platforms and reconcile text, fields and values. Link defects to actual candidate builds; update help/limitations. UAT acceptance cannot be faked by marking your own PR merged.

Deliver: fixed PRs, parity report, help content and limitations list. **Pass when:** React and Flutter display same business results; all accepted limitations have backlog ticket.

Sprint 8 / Backend and API

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S8-API-01

Build: Harden FastAPI security: rate limits, CORS, token rotation/revocation, authorization tests, request size limits and secure headers.

How to implement alone: Configure rate/request limits, allowed origins, secure headers and verified token lifecycle. Run cross-user checks for every sensitive endpoint. Keep necessary settings explicit and secrets out of error/log output.

Deliver: security PR, test evidence and configuration guide. **Pass when:** Critical authorization tests pass; excessive/invalid requests are safely rejected and audited.

S8-API-02

Build: Add observability and deployment: structured logs, metrics, tracing, health/readiness dependencies, staging/prod config and rollback procedure.

How to implement alone: Build an immutable API image, record its digest/version, deploy with secrets and dependency-aware readiness. Instrument safe logs/metrics/traces and rehearse rollback with schema compatibility. Record environment-specific configuration without values of secrets.

Deliver: observability middleware, dashboards/queries, deployment and rollback runbooks. **Pass when:** A test request can be traced through API, data and AI; failed dependency makes readiness fail but liveness remain healthy.

S8-API-03

Build: Run API integration/load/reliability tests for login, portfolio, chat, paper trading, alerts, goals and spending; resolve bottlenecks assigned.

How to implement alone: Exercise all included routes under bounded, documented load and provider failure. Report environment, concurrency, latency percentiles and errors without claiming enterprise scale from a small smoke. Fix critical reliability regressions and retest.

Deliver: load scripts, results, fixed PRs and capacity note. **Pass when:** No data corruption under test; critical p95/timeout goals are met or documented with accepted mitigation.

Sprint 8 / Data platform and quality

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S8-DAT-01

Build: Implement Unity Catalog permissions and environment separation for identity, portfolio, AI, trading and spending data; validate least privilege.

How to implement alone: Apply least-privilege roles for operational database, catalog and workloads. Validate denied access as well as allowed queries. Logical free-tier schemas are not equivalent to physically isolated production environments.

Deliver: access-control script, matrix and validation evidence. **Pass when:** Web/mobile never access Databricks directly; service identities can access only required schemas; denied tests pass.

S8-DAT-02

Build: Orchestrate production Databricks workflows for market ingestion, RAG ingestion, portfolio snapshots, alert evaluation, paper snapshots and spending summaries with retries/alerts.

How to implement alone: Configure market/document ingestion, snapshots, alerts and summaries as versioned workflows with bounded retries and monitoring. Confirm permissions, resource/cost bounds and no overlapping duplicate runs. A schedule definition alone is not proof of a successful job.

Deliver: workflow definitions, schedule matrix, retry policy and failed-run demo. **Pass when:** Upstream failure prevents unsafe downstream execution; retries are bounded; owners receive actionable failure context.

S8-DAT-03

Build: Run final data reconciliation, retention, backup/export and deletion verification across user, portfolio, trading, AI and spending domains.

How to implement alone: Run final reconciliation and real restore/export/deletion checks on controlled records. Document retention exceptions and analytical-copy deletion behaviour. Protect backups and prove recovery; an untested backup file is not sufficient.

Deliver: final data quality report, retention/deletion matrix and recovery rehearsal evidence. **Pass when:** Critical DQ checks pass; account deletion/export behavior is verified; recovery steps are documented.

Sprint 8 / AI, reasoning and evaluation

All tasks owned by Shivraj. Source IDs are unchanged. Operational records belong in Postgres; Delta is analytical/history storage. Detailed provider capability must be checked in your account.

S8-AI-01

Build: Run full AI regression across education, risk, comparison, RAG, paper trading, portfolio, goals and spending; fix prompt/tool failures assigned to the cohort.

How to implement alone: Run the combined regression suite on exact released model/prompt/retriever/source versions across all eight AI domains. Separate numerical, retrieval, citation and safety outcomes. Do not promote a candidate with unresolved critical privacy/factuality failures.

Deliver: master evaluation suite, regression report and fixed prompt/tool PRs. **Pass when:** No critical safety failure; thresholds are met or explicitly accepted with owner and mitigation.

S8-AI-02

Build: Register AI experiments/models/prompts in MLflow, define version tags, promote approved versions and create latency/error/quality monitoring queries.

How to implement alone: Log approved experiments/prompt/model/source versions in available MLflow/registry facilities. Record which identifiers are actually used by deployment and how to roll back. Monitor quality/latency/errors with safe aggregate telemetry.

Deliver: MLflow records/model registry entries, promotion checklist and monitoring dashboard queries. **Pass when:** Every production AI component has owner, version, dataset, metrics and rollback target.

S8-AI-03

Build: Execute red-team tests for prompt injection, unsupported advice, stale data, fabricated citation, privacy leakage and cross-user portfolio requests.

How to implement alone: Red-team instruction injection in sources, fabricated citations, stale financial claims and cross-user questions. Record failed case, remediation and retest. Token/model access or source changes require rechecking affected evaluation cases.

Deliver: red-team report, failing-case tickets and retest evidence. **Pass when:** All critical attacks are blocked or safely limited; logs contain no secret or other-user data.

12 / Security, privacy and real-data safeguards

Identity and authorization

Verify provider identity and token issuer/audience/expiry using the chosen OAuth/session design. Enforce ownership on profiles, portfolios, conversations, trades, goals, uploads and notifications in server services/repositories. Test one user attempting another user record, including indirect parent/child IDs.

Secrets, uploads and logs

Store credentials outside source and client builds; keep example files to names/non-sensitive defaults. Limit request/upload sizes and allowed formats. Mask sensitive descriptions before optional model calls. Logs use safe identifiers and correlation IDs, not tokens, raw statements or unredacted private prompts.

AI trust boundary

Retrieved documents and uploaded text are untrusted data. They cannot alter tool permissions or instruct the app to expose another account. Validate tool arguments and cited IDs, keep deterministic metrics outside the LLM and return insufficient evidence when support is absent. Track model/prompt/retriever/source versions for every evaluated candidate.

Financial integrity

Use transactional storage for order effects, scoped request idempotency and safe concurrency control. Unknown network outcome is not proof that execution failed. Reconcile records independently and use traceable correction entries. An unreconciled ledger is a release blocker, even when the UI looks correct.

Data access and redistribution

Downloaded public data is not automatically licensed for every public display or redistribution. Verify source terms and selected account access before publication. Cache only as permitted and keep true observation timestamps. Do not work around source access restrictions.

Release-blocking examples

Cross-user data leakage; leaked credential; fabricated source/citation presented as real; duplicate/partial order; incorrect accepted calculation; provider outage silently replaced with fake values. Fix the defect or withdraw the affected feature, then rerun relevant checks.

13 / Deployment and operations runbook

1. Verify the chosen account capabilities

The handbook names Azure Static Web Apps, Azure Container Apps, GHCR, Databricks/Lakebase/Vector Search/MLflow and FCM. Treat these as source architecture, not a current free-tier guarantee. Record actual access, limits, regions, required permissions and cost/budget settings. Do not create a complex second infrastructure stack to hide an unavailable service.

2. Build versioned artifacts

Run backend/client checks and generate contract clients from the accepted version. Produce the API container and record immutable image digest; produce web bundle and supported Android artifact. iOS build claims require actual tooling/signing evidence. Keep source commit and artifact IDs linked.

3. Configure and migrate

Supply backend secrets through approved environment mechanisms; frontend/mobile receive public configuration only. Run reviewed migrations with tested recovery. Confirm required services/readiness and permissions before opening access. Keep demo/test schemas distinguishable from any sensitive operational data.

4. Smoke and roll back

Test login, real quote, holdings, chat citation and included paper/alert/goal/spending operations through the actual deployed endpoint. Rehearse rollback to the previous working artifact and document schema compatibility; rolling back code alone may not undo a migration. Verify a controlled backup restore/reset.

5. Observe jobs and quality

Monitor request errors/latency, provider failures/cache age, ingestion counts, DQ breaks, model/retrieval failures and alert delivery attempts. Bound job retries and avoid overlapping duplicate work. Separate user-visible failure from retry scheduling so a stuck job does not appear complete.

No false production claims

A private educational demo with logical free-tier environments is not an enterprise-isolated regulated production system. Report actual test environment and load. Record any local-only/cloud-unavailable component as a limitation rather than renaming it deployed.

14 / Acceptance, evidence and project submission

Definition of Ready

Approved user story and business rule; frozen required contracts/examples; units/currency/time definitions; accepted predecessors or explicitly approved fixture path; named implementation owner and real acceptance route; no unresolved deterministic calculation decision.

Definition of Done

Implemented code/notebook/migration; relevant unit/contract/integration/UI/DQ/AI tests; no secrets/private data in evidence; both intended consumers verified; versioned handoff and rollback note; actual deployment/demo proof; acceptance recorded or remaining review clearly pending.

What to measure

Use source expectations: S1 policy set of at least 30 prompts, S2 at least 12 risk profiles, S3 comparison/factuality examples, S4 at least 40 representative questions and recall@k/filter/citation checks, S5 reconciliation/concurrency, S6 cooldown/scenario fidelity, S7 goal/category/trend/row totals, S8 full regression and red-team. Record denominators and versions; an unrun test is not passed.

Suggested board fields

Task ID; owner Shivraj; source section; status; dependency IDs; estimated/actual hours; contract/schema/source version; branch/commit; run command; expected/actual; evidence link; blocker; limitation; acceptance authority/result. States: Backlog, Ready, In progress, Blocked, Review, Accepted, Deferred.

Final internship package

• Source repository and accepted revision/artifacts, setup/API/schema guides and environment variable names without secrets.

• Real-data source manifest, import/DQ logs, RAG source/evaluation versions and metric/formula definitions.

• Web/mobile recordings, accepted/deferred task ledger, tests/UAT and reproducible demo personas.

• Architecture/presentation, deployment/restore/rollback runbook, known limitations and next-phase backlog.

Explain your own contribution accurately: you implemented the solo work. AI assistance may be documented, but you must be able to explain the code, data boundaries, calculations and tests. Do not attribute reviews or implementation to uninterested teammates.

15 / Source references and first actions

Authoritative uploaded project source

FinEx_Combined_Implementation_Handbook_v1 (1)(2).pdf, Version 1.0, 10 August 2026, supplied in this conversation. Sections 1-4 define architecture/ready/done/Week 0; Section 5 gives the roadmap; Sections 6.1-6.8 contain the 96 sprint cards; Sections 7-8 define handoff and prototype guardrails. All 96 source IDs are represented in this blueprint.

Real-data and documentation references

[R1] NSE official market-report listing

[R2] NSE data sharing and usage policy

[R3] yfinance documentation and data-use notice

[R4] Alpha Vantage access/limits - optional alternative only

[R5] SEBI investor educational reading material

These public references were checked on 16 October 2026 in the preceding planning work. NSE lists downloadable reports, yfinance documents personal-use constraints and SEBI provides educational materials. Account capabilities, endpoint coverage and permissions still need confirmation before use. This blueprint does not claim that a provider key, dataset download or cloud deployment has been tested.

Your next three actions

• Run the actual repository and fill the working/broken/missing inventory. Attach real command results.

• Create the 96 source task IDs under your name and mark only genuinely verified work complete. Start with Week 0/S1 prerequisites, not all features at once.

• Acquire one permitted real market record and three approved real documents. Preserve their source/date/hash and plan the first end-to-end data/citation proof.

Completion and forecast

The full scope remains intact. 10 November is a bounded demonstration checkpoint with explicit deferrals, not a promise that a twelve-developer handbook can be completed solo in 26 days. After S1, estimate remaining cards using measured throughput and publish a separate full-completion forecast.